Introduction to Algorithmic Complexity



Lecture 1 — Preliminary Concepts

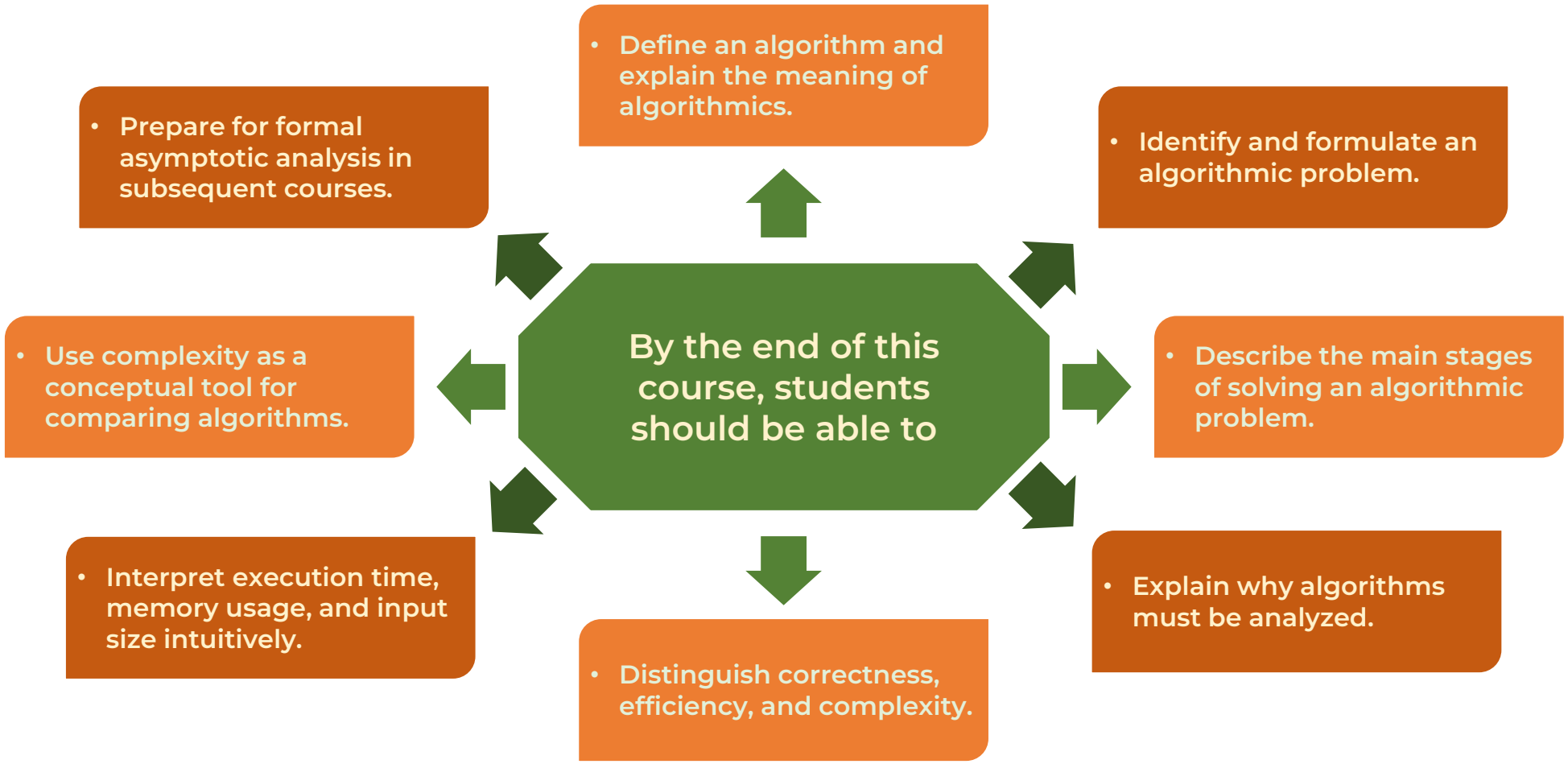
Abdelkrim Mebarki
Department of Computer Science

University of Science and Technology of Oran – Mohamed Boudiaf (USTOMB)

Academic Year : 2026 - 2027

https://sites.google.com/view/a-mebarki/

Learning Objectives



What Is an Algorithm?

The concept

- An algorithm is a finite, well-defined sequence of steps that transforms input data into the required output.

What is it for ?

- An algorithm provides a procedure for solving a class of problems—not just one particular instance.

Example:

- Finding the maximum value in an array.
- Input: an array A of n numbers.
- Output: the largest value contained in A.
- Procedure: inspect the elements and keep track of the largest seen so far.

Abstraction level

- An algorithm should be independent of a particular programming language.

Characteristics of an Algorithm

Finiteness



The procedure must terminate after a finite number of steps.

Definiteness / Precision



Every step must be unambiguous and precisely specified.

Effectiveness



Each operation must be basic enough to be carried out in finite time.

Input



Zero or more values supplied to the algorithm.

Output



One or more results produced by the algorithm.

Determinism



Given the same input and the same conditions, a deterministic algorithm follows the same sequence of decisions.

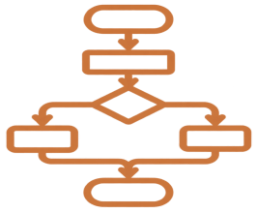
Example: Maximum of an Array

-
```
MAXIMUM(A)
    max ← A[0]
    for i ← 1 to length(A)-1 do
        if A[i] > max then
            max ← A[i]
    return max
```

- **Key idea**
  1. The algorithm terminates because the loop has a finite number of iterations.
  2. Each step is precisely defined.
  3. It works for every valid non-empty input array.
  4. The same input produces the same result.
  5. We will later ask: how does its cost grow as n grows?

Algorithm vs. Program vs. Software

Algorithm



abstract method for solving a problem.
Example: *Binary Search*

Program



implementation of an algorithm in a programming language.
Example : *a C++ implementation of binary search*

Software



a broader system including programs, libraries, configuration, interfaces, documentation, and other components.
Example : *a search application that uses binary search together with a user interface, data storage, logging, etc.*

The same algorithm can be implemented by different programs and in different languages.

What Is Algorithmics?



Is the study of algorithms:
Their **design**, **correctness**, **efficiency**, **limitations**, and **applications**.



It addresses **questions** such as:

- ❑ How can we solve this problem?
- ❑ Can we solve it correctly?
- ❑ How efficiently can we solve it?
- ❑ How does the cost change when the input becomes larger?
- ❑ Can a better algorithm be designed?



Algorithmics therefore **connects**

- ✓ **problem solving**,
- ✓ **data structures**,
- ✓ **mathematical analysis**,
- ✓ and **implementation**

The Algorithmic Problem

- An **algorithmic problem** specifies what must be computed from given input data.
- A problem statement generally defines:



→**Input specification** : what data are provided?

→**Output specification** : what result is required?

→**Constraints** : what values or sizes are allowed?

- Example – Sorting
  - Input: a sequence of n numbers.
  - Output: the same numbers arranged in nondecreasing order
- *One problem can have many different algorithms.*

Problem vs. Problem Instance



Problem — the general computational task.

Problem instance — one concrete input belonging to that problem.



Example: Sorting

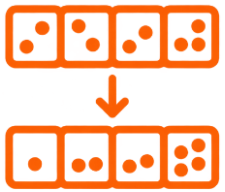
Problem: sort an arbitrary sequence of numbers.
Instance: [7, 2, 9, 4, 1].
Another instance: [10, 3, 3, 8, 5, 1, 12].

The algorithm should solve **all valid instances**, not just one example.
This distinction is essential for complexity analysis because input size varies from one instance to another.

Examples of Algorithmic Problems



Searching
Does a value x occur in an array?



Sorting
Rearrange values according to a specified order.



Maximum / Minimum Find the largest or smallest value.



Shortest Path
Find a minimum-cost path between two vertices in a graph.



String Matching Find occurrences of a pattern in a text.



Matrix Multiplication
Compute the product of two matrices.

Each problem may admit several algorithms with very different performance.

Solving an Algorithmic Problem

1. Analyze the problem.
2. Identify the input data.
3. Specify the required output.
4. Identify constraints and assumptions.
5. Design an algorithm.
6. Prove or argue correctness .
7. Implement the algorithm.
8. Test and validate the implementation.
9. Analyze performance and improve the solution if necessary.

A disciplined workflow helps separate problem understanding from implementation.

Step 1 — Analyze the Problem



- Before writing code
  - **Determine exactly what is being asked.**
- Example: Find the largest value in an array.
  - Questions:
    - Is the array empty?
    - Are values integers, real numbers, or arbitrary comparable objects?
    - What should happen if several elements have the same maximum?
    - What is the expected output?
- Poorly specified problems → Incorrect implementations
  - Even when the code itself is syntactically correct.

Step 2 — Input, Output, and Constraints

- Example: Search for x in an array A.
  - **Input**: A[0..n−1] and target value x.
  - **Output**:
    - Index i such that A[i] = x,
    - Special value such as −1 if x is absent.
  - **Constraint**: n ≥ 0.



- Constraints matter because they influence algorithm selection.
  - Example:
    - for n ≤ 20,000,000, a linear scan may be acceptable;
    - for repeated queries on a large static dataset, preprocessing and faster search may be worthwhile.

Step 3 — Design the Algorithm



- Design the solution
  - At an **abstract level**
  - Before committing to implementation details.
  - **At this stage, focus on logic—not syntax !**
- Example: Linear search
  - Start from the first element.
  - Compare each element with x.
  - Stop when x is found.
  - If the end is reached, report that x is absent.

Step 4 — Correctness, Implementation, Testing

- **Correctness**
  - Does the algorithm always produce the required result for every valid input?
- **Implementation**
  - Translate the algorithm into a programming language while preserving its logic.
- **Testing**
  - Execute the implementation on representative cases.

Useful test cases

- Typical input.
- Smallest valid input.
- Largest or boundary values.
- Empty input when allowed.
- Repeated values.
- Adversarial or worst-case patterns.



  - *Testing provides evidence; formal reasoning provides stronger guarantees.*

Correctness ≠ Efficiency ≠ Complexity



Correctness

Whether the algorithm produces the right answer.



Efficiency

How economically it uses computational resources.



Complexity

Formal way of describing how resource requirements grow with input size.

- An algorithm can be correct but inefficient.
  - First question: Does it solve the problem correctly?
  - Second question: How well does it scale?

Why Analyze Algorithms?

- **Different algorithms** can solve **the same pro**blem.
- Example → **searching in a sorted collection**:
  - **Linear search** examines elements one by one.
  - **Binary search** repeatedly halves the search interval.
- The difference becomes dramatic as the **input grows**.
- Algorithm analysis helps us choose solutions
  - **That remain practical at scale**.
- Important resources:
  - Time — computation / execution cost.
  - Space — memory required.

Goal: understand **scalability**, not merely measure one execution.

Same Problem, Different Algorithms

- **Problem**: Find x in a sorted array of n elements ?
- **Linear Search**:
  - May inspect elements sequentially.
  - Can stop early if x is found.
- **Binary Search:**
  - Inspect the middle element.
  - Discard half of the remaining search space.
  - Repeat until x is found or the interval becomes empty.

Both are correct → Their **growth in work** is fundamentally different.

Input Size Matters

- Algorithm performance depends strongly on **the input size**.
- Consider searching an array:
  - n = 10 → at most 10 positions for linear search.
  - n = 1,000 → at most 1,000 positions.
  - n = 1,000,000 → at most 1,000,000 positions.
- An algorithm that is acceptable for **small inputs** may become unusable for **large inputs**.
- Complexity analysis studies this growth systematically.



Execution Time: A First Intuition

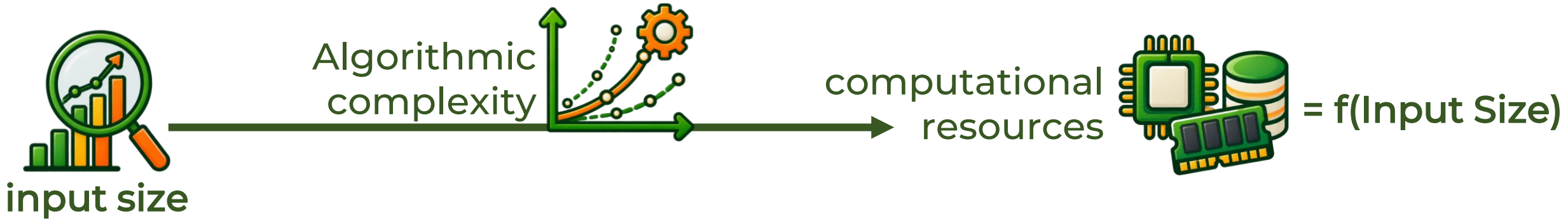
- Suppose one operation takes approximately **1 microsecond**.
- An algorithm performing:
  - **1,000** operations → about **1 ms**.
  - **1,000,000** operations → about **1 s**.
  - **1,000,000,000** operations → about **1,000 s ≈ 16.7 min**.



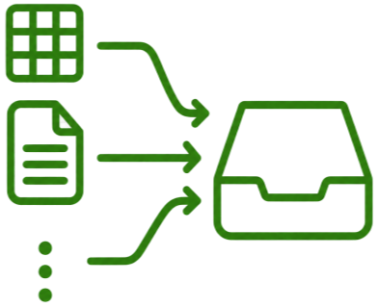
- This is only an illustrative model
  - Real execution times depend on hardware, language, memory hierarchy, compiler, and implementation.
- Complexity analysis abstracts away most **machine-specific details** and focuses on growth.

Introduction to Algorithmic Complexity

- **Algorithmic complexity** describes the computational resources required by an algorithm as **a function of input size**.
- The two fundamental resources introduced here are:
  - **Time complexity** — how the amount of computation grows.
  - **Space complexity** — how memory requirements grow.
- Complexity is **primarily about growth**, rather than an exact stopwatch measurement.



What Is Input Size?



- **Input size**, usually denoted by **n**,
  - measures how much information the algorithm must process.
- Examples:
  - Array: **n = number of elements**.
  - String: **n = number of characters**.
  - Graph: size may involve both **|V| vertices** and **|E| edges**.
  - Matrix: **dimensions m × n** may be more informative than a single n.
- Choosing the right size parameter is an essential part of complexity analysis.

Elementary Operations

- To reason about running time, we count basic **computational steps**.
- Examples of elementary operations:
  - **Assignment**: `x ← y`
  - **Comparison**: `x < y`
  - **Arithmetic operation**: `x + y`
  - **Array access**: `A[i]`
- Instead of measuring seconds directly, we can estimate **how the number of such operations changes with n**.
- The exact choice of what counts as elementary depends on the **computational model**.

Time Complexity — Intuition

- **Time complexity**
  - Measures how the amount of computation performed by an algorithm depends on input size.
- Example → **summing n numbers**:
  - Each element must be processed once.
  - The number of iterations grows proportionally with n.
- Example → **comparing every pair among n elements**:
  - The number of pairs grows much faster than n.

Space Complexity — Intuition

- **Space complexity**
  - Describes how memory requirements depend on input size.
- Example 1 → **Find the maximum in an array**
  - Only a few variables are needed in addition to the input.
- Example 2 → **Create a 2nd array containing n transformed values**
  - Additional storage grows with n.
- Important distinction:
  - **Input space** — memory occupied by the input.
  - **Auxiliary space** — extra memory used by the algorithm.
- The convention used should always be stated clearly

A Simple Comparison

- Problem: **Compute the sum of n values**.
- **Algorithm A** – Scan the array and accumulate the sum.
  - Time grows with n.
  - Extra memory can remain constant.
- **Algorithm B** – Repeatedly recompute partial sums from the beginning.
  - Much more work is performed as n grows.
  - It may still produce the correct answer.
- Correctness alone is not enough
  - Two correct algorithms can have very different scalability.

Example — Searching an Array

- **Problem**
  - Given an array A and a target x, determine whether x occurs in A.
- Two basic solutions:
  - Algorithm 1 – **Linear Search**.
  - Algorithm 2 – **Binary Search** — Requires the array to be sorted.
- **Question**
  - Which one should we use?
- The answer depends on input properties, number of queries, preprocessing cost, and required performance.

Linear Search

```
LINEAR-SEARCH(A, x)
    for i ← 0 to length(A)-1 do
        if A[i] = x then
            return i
    return -1
```

A more detailed and rigorous version of the algorithm is provided in the course lecture notes.

Key idea

→ Works on unsorted arrays.

→ Checks elements sequentially.

→ **Best case**

- x is found immediately.

→ **Worst case**

- x is last—or absent.

→ **Key question**

- How many elements may need to be inspected when n grows?

Binary Search

```
BINARY-SEARCH(A, x)
 low ← 0
 high ← length(A)-1

 while low ≤ high do
   mid ← floor((low + high)/2)
   if A[mid] = x then return mid
   if A[mid] < x then low ← mid + 1
   else high ← mid – 1

 return –1
```

A more detailed and rigorous version of the algorithm is provided in the course lecture notes.

Key idea

→ Requires A to be sorted.

→ Each comparison eliminates roughly half of the remaining candidates.

→ The search interval

- Shrinks very rapidly.

→ **Key idea**

- exploit structure in the input.

→ This leads naturally to logarithmic growth.

Linear vs. Binary — Intuitive Comparison

- Suppose **n = 1,000,000**.
  - →Linear search may inspect up to **1,000,000** elements.
  - →Binary search needs only about **20** halving steps because **2^{20} = 1,048,576**.
- Both algorithms solve the same search problem,
  - →but their scalability is radically different.
- **Important condition**
  - Binary search is applicable because **the data are sorted**.
- Algorithm analysis helps make this difference visible **before implementation**.

Best/Worst Case, Typical Behavior

- **Best case** → The most favorable input or execution scenario.
- **Worst case** → The most costly valid input or execution scenario.
- **Average / expected case**

  →Performance averaged under an assumed input distribution or randomness model.
- Linear search example:
  - **Best case**: x is the first element.
  - **Worst case**: x is absent or at the last position.
- Why worst-case analysis is useful?
  - It provides a guarantee on the maximum amount of work required.

From Intuition to Asymptotic Analysis

- **Asymptotic analysis**
  - It studies how resource usage grows as the input size becomes large.
- We focus on the dominant growth behavior.

- Examples of growth patterns:
  - **Constant** — Roughly independent of n.
  - **Logarithmic** — Grows very slowly with n.
  - **Linear** — Grows proportionally to n.
  - **Quadratic** — Grows roughly as n^2.

What We Have Learned

- An **algorithm** is a precise, finite procedure for solving a computational problem.
- A **problem** describes a general task; an **instance** is a concrete input.
- **Solving a problem** involves analysis, design, correctness, implementation, testing, and validation.
- Several algorithms may **solve the same problem**.
- **Correctness** does not imply **efficiency**.
- **Complexity** captures how computational resource requirements grow with input size.
- **Time** and **space** are the two fundamental resources introduced here.
- **Asymptotic analysis** provides foundation for formal comparison.

Take-Home Questions

1. Can two different algorithms always be compared using only execution time on one machine?
2. What is the difference between a problem and an instance?
3. Give an example of a correct but inefficient algorithm.
4. Why is input size important in algorithm analysis?
5. What is the difference between time complexity and space complexity?
6. Why can binary search outperform linear search so dramatically?
7. What condition must hold for binary search to be applicable?